We wrote the `buildTable` method using recursive form. As explained in appendix E, this allows us to provide cleaner and more easily understandable code, but in some languages concrete implementations can be more performant when implemented using explicit iterations.

We pass 3 arguments to the method: a `TreeNode node`, that is the current node in the traversal of the tree, a sequence, that is the path from the root to current node (where we add a 0 for a “left turn” and a 1 for a “right turn”) and the `Map` that will hold the associations between characters and bit sequences.

At line #1, we check if the set of characters in the node has only one character. If it does, it means we have reached a leaf, and so the recursion can stop: the bit sequence that is associated with the character in the node is the path from root to current node, stored in the `sequence` variable.

Otherwise we check if the node has left and right children (it will have at least one, because it’s not a leaf) and traverse them. The crucial point here is how we build the sequence argument in the recursive calls: if we traverse the left child of current node, we add a 0 at the end of the sequence, while if we traverse the right child, we add a 1.

Table 2.5 shows the compression table produced starting from the tree shown in figure 2.15; the last column would not be part of the actual compression table, but it’s useful to understand how most used characters end up translated into shorter sequences (which is the key to an efficient compression).